

Operating Systems - Question Bank (CIE1)

RV University | CS1103 | Semester II

200+ MCQs · Fill in the Blanks · True/False · One-Word Answers

All questions are based on Unit I (Introduction to OS) and Unit II (Process Management).

Answer Key appears at the end of each section.

Table of Contents

- **Part A: Multiple Choice Questions (MCQ)**
 - **Section 1: OS Definitions, Types & Functions**
 - **Section 2: OS Services, Structure & Booting**
 - **Section 3: System Calls & Virtualization**
 - **Section 4: Processes & PCB**
 - **Section 5: Process State Models**
 - **Section 6: Threads & IPC**
 - **Section 7: Schedulers & Scheduling Criteria**
 - **Section 8: Scheduling Algorithms — Conceptual**
 - **Section 9: Scheduling Algorithms — Numerical**
 - **Part B: Fill in the Blanks**
 - **Part C: True or False**
 - **Part D: One-Word / One-Line Answers**
 - Master Answer Key
-

PART A: MULTIPLE CHOICE QUESTIONS

Section 1: OS Definitions, Types & Functions

Q1. An Operating System acts as an intermediary between:

- **A)** Application programs and the network
- **B)** The user and the computer hardware
- **C)** The CPU and the RAM only
- **D)** The kernel and device drivers

Q2. Which of the following BEST defines the kernel of an OS?

- **A)** The entire suite of OS programs
- **B)** A system utility program used for compiling code
- **C)** The one program running at all times on the computer; the core of the OS
- **D)** The outermost layer of the OS that interacts with users

Q3. Which of the following is NOT a category of software in an OS-managed system?

- **A)** Kernel
- **B)** System Programs
- **C)** Application Programs
- **D)** Microcode Programs

Q4. The OS analogy used in the course material describes it as similar to:

- **A)** A teacher
- **B)** A government
- **C)** A factory
- **D)** A library

Q5. A Mainframe OS typically offers which THREE kinds of services?

- **A)** Compiling, Linking, Loading
- **B)** Batch, Transaction Processing, Timesharing
- **C)** Real-time, Interactive, Distributed
- **D)** Scheduling, Swapping, Paging

Q6. Which OS is cited as an example of a Mainframe OS?

- **A)** Android
- **B)** TinyOS
- **C)** OS/390
- **D)** VxWorks

Q7. Transaction-processing systems (e.g., check processing at a bank) handle:

- **A)** A few very large jobs per day
- **B)** Hundreds or thousands of small requests per second
- **C)** Only batch jobs overnight
- **D)** Single-user interactive tasks

Q8. Which type of OS is characterized by software stored entirely in ROM with no user-installed programs allowed?

- **A)** Server OS
- **B)** Real-Time OS
- **C)** Embedded OS
- **D)** Handheld OS

Q9. TinyOS is primarily designed for:

- **A)** Mainframe computers
- **B)** Sensor-node networks
- **C)** Desktop personal computers
- **D)** Multiprocessor systems

Q10. A Hard Real-Time OS differs from a Soft Real-Time OS in that:

- **A)** Hard RTOS runs on embedded hardware only
- **B)** Hard RTOS must provide absolute guarantees that actions occur by a certain time
- **C)** Soft RTOS has no timing constraints at all
- **D)** Hard RTOS is cheaper to implement

Q11. Smartphones are an example of which type of RTOS?

- **A)** Hard real-time
- **B)** Soft real-time
- **C)** No real-time characteristics
- **D)** Batch OS

Q12. Smart card OSes are characterized by:

- **A)** High computing power and large RAM
- **B)** Running standard desktop applications
- **C)** Very severe processing power and memory constraints
- **D)** Requiring constant user interaction

Q13. Some smart cards hold a ROM with an interpreter for:

- **A)** Python Virtual Machine
- **B)** .NET Common Language Runtime
- **C)** Java Virtual Machine (JVM)
- **D)** LLVM IR interpreter

Q14. Which OS is NOT an example of a sensor-node OS?

- **A)** TinyOS
- **B)** Contiki
- **C)** RIOT
- **D)** macOS

Q15. The primary objective of OS job scheduling is:

- **A)** Maximize power consumption
- **B)** Minimize disk space
- **C)** Allocate computer resources among competing processes fairly and efficiently
- **D)** Ensure all programs are compiled successfully

Q16. A Personal Computer OS primarily provides:

- **A)** Services for many concurrent network users
- **B)** Good support for a single user with multiprogramming
- **C)** Industrial process-control guarantees
- **D)** Transaction processing for banks

Q17. Multiprocessor operating systems are also called all of the following EXCEPT:

- **A)** Parallel computers
- **B)** Multi-computers
- **C)** Multiprocessors
- **D)** Multitasking single-core computers

Q18. Which pair of operating systems is an example of Handheld Computer OS?

- **A)** Linux and Windows
- **B)** Solaris and FreeBSD
- **C)** Android and iOS
- **D)** VxWorks and QNX

Q19. Which OS feature makes embedded systems simpler to design than general-purpose OSes?

- **A)** They support downloading of any application
- **B)** No need for protection between applications since no untrusted software runs
- **C)** They run on the largest available hardware
- **D)** They support full virtual memory management

Q20. Sensor-node OSes are event-driven primarily because:

- **A)** Users frequently interact with them via GUI
- **B)** They respond to external events or make measurements based on an internal clock with limited RAM and battery
- **C)** They support real-time graphics rendering

- **D)** They are designed for server environments
-

Section 2: OS Services, Structure & Booting

Q21. Which of the following is the FIRST action performed during system boot?

- **A)** OS kernel is loaded into memory
- **B)** The bootstrap program initializes CPU registers, device controllers, and memory
- **C)** The init process is started
- **D)** User programs are loaded

Q22. Cold booting refers to:

- **A)** Restarting using the OS restart function
- **B)** The computer being completely switched off and then turned on
- **C)** Rebooting after a crash
- **D)** Hibernating the system

Q23. A Hardware Abstraction Layer (HAL):

- **A)** Directly controls the user interface
- **B)** Abstracts access to different hardware peripherals with a common set of APIs
- **C)** Provides file system encryption
- **D)** Manages virtual memory allocation

Q24. OS provides a uniform interface for I/O devices so that programmers can access different devices using:

- **A)** Device-specific unique commands for each hardware
- **B)** Simple reads and writes regardless of device type
- **C)** Only via privileged assembly language calls
- **D)** Direct manipulation of device registers

Q25. The "Resident Set" of a program consists of:

- **A)** The program's documentation and config files
- **B)** The program's Code, Data, and Stack that must be in main memory for execution
- **C)** The program's source code on disk
- **D)** The process's open file handles

Q26. Which OS service collects usage statistics for resources and monitors performance parameters?

- **A)** Error Detection
- **B)** File Management
- **C)** Accounting

- **D) Security**

Q27. What does the OS read a PPT file as when loading it into memory?

- **A) A structured XML tree**
- **B) A stream of bytes**
- **C) A sequence of binary instructions**
- **D) A compressed archive**

Q28. The UNIX operating system has which type of structure?

- **A) Layered microkernel**
- **B) Monolithic**
- **C) Client-server**
- **D) Exokernel**

Q29. In a monolithic OS structure, all OS components:

- **A) Run as separate server processes**
- **B) Are combined into one large executable running entirely in kernel mode**
- **C) Are isolated in user-space libraries**
- **D) Communicate via message passing only**

Q30. Which statement about the OS as a control mechanism is CORRECT?

- **A) The OS never relinquishes control of the processor**
- **B) The OS is external hardware separate from the programs it controls**
- **C) The OS functions like ordinary computer software — it is a program executed by the processor**
- **D) The OS permanently occupies the CPU**

Q31. The kernel contains:

- **A) Only device driver programs**
- **B) The most frequently used functions of the OS, always resident in main memory**
- **C) Only the file system code**
- **D) User application data**

Q32. A hardware interrupt is triggered by:

- **A) A software bug in the user program**
- **B) A system call instruction**
- **C) A hardware device sending a signal to the CPU via the system bus**
- **D) The OS scheduler timer only**

Q33. During interrupt handling, at step 3, the assembly language routine:

- **A) Resets the CPU**
- **B) Saves all CPU registers**
- **C) Calls the C service routine directly**

- **D)** Loads the new process's PC

Q34. After an interrupt is handled, the scheduler:

- **A)** Always resumes the interrupted process
- **B)** Always loads a new process
- **C)** Selects a process to run next, which may or may not be the interrupted one
- **D)** Terminates the interrupted process

Q35. OS error detection includes all of the following EXCEPT:

- **A)** Hardware device failure detection
- **B)** Division by zero detection
- **C)** Preventing file renaming to an already-existing name
- **D)** Compiling user programs for correctness

Q36. Resource allocation by the OS resolves:

- **A)** Syntax errors in programs
- **B)** Conflicts between competing resource requests from multiple processes
- **C)** Disk formatting operations
- **D)** Network packet routing conflicts

Q37. Which layer is immediately below the OS in the layered abstract view?

- **A)** Application Programs
- **B)** User Interface
- **C)** Computer Hardware (CPU, Memory, I/O devices)
- **D)** System Programs

Q38. A device controller is responsible for:

- **A)** Executing user program instructions
- **B)** Moving data between peripheral devices and its local buffer storage
- **C)** Translating virtual to physical addresses
- **D)** Scheduling CPU processes

Q39. To ensure orderly access to shared memory between CPU and device controllers, a _____ is used.

- **A)** Cache controller
- **B)** Memory controller
- **C)** DMA engine
- **D)** Bus arbitrator (in this course context)

Q40. Which service provides editors and debuggers to assist programmers?

- **A)** Program Execution
- **B)** File Management
- **C)** Program Development

- **D) Security**
-

Section 3: System Calls & Virtualization

Q41. A system call is best described as:

- **A) A function call between two user programs**
- **B) A request from a user-level process to the kernel to perform a specific privileged task**
- **C) A hardware interrupt from a device**
- **D) A compiler directive for OS interaction**

Q42. When a user program invokes a system call, the CPU mode switches from:

- **A) Kernel Mode to User Mode**
- **B) User Mode to Kernel Mode**
- **C) Supervisor Mode to Kernel Mode**
- **D) Real Mode to Protected Mode**

Q43. The mechanism that causes a CPU mode switch during a system call is called a:

- **A) Hardware interrupt**
- **B) Trap**
- **C) Context switch**
- **D) Page fault**

Q44. Standard input (keyboard) is associated with which file descriptor number?

- **A) 0**
- **B) 1**
- **C) 2**
- **D) 3**

Q45. Standard output (screen) is associated with which file descriptor number?

- **A) 0**
- **B) 1**
- **C) 2**
- **D) 3**

Q46. Standard error is associated with file descriptor:

- **A) 0**
- **B) 1**
- **C) 2**
- **D) -1**

Q47. In the `write(1, message, sizeof(message))` system call, the first argument 1 represents:

- **A)** One byte to write
- **B)** The file descriptor for standard output
- **C)** The priority level
- **D)** The process ID

Q48. Which system call is used in UNIX to create a new process?

- **A)** `create()`
- **B)** `new()`
- **C)** `fork()`
- **D)** `spawn()`

Q49. Which system call category does `getpid()` belong to?

- **A)** Process Control
- **B)** File Management
- **C)** Information Maintenance
- **D)** Communication

Q50. The `strace` tool in Linux is used to:

- **A)** Compile programs with debug symbols
- **B)** Monitor and tamper with system calls made by a process
- **C)** Display memory usage of a process
- **D)** Show network activity of a process

Q51. `strace -c ls` displays:

- **A)** A line-by-line trace of each system call
- **B)** A count of system calls with timing information
- **C)** The source code of `ls`
- **D)** Memory map of `ls`

Q52. Why are applications OS-specific?

- **A)** Because each OS uses different hardware
- **B)** Because each OS provides a unique set of system calls
- **C)** Because different OSes use different CPU architectures
- **D)** Because of network protocol differences

Q53. Which of the following is NOT a valid solution for making an application portable across OSes?

- **A)** Writing in an interpreted language like Python
- **B)** Using a virtual machine language like Java
- **C)** Compiling to machine code directly without any abstraction
- **D)** Using a standard API where compiler generates OS-specific binaries

Q54. Virtualization technology creates the illusion that each VM:

- **A)** Shares the CPU with all other VMs at the same time
- **B)** Runs on its own private computer
- **C)** Has access to the host OS kernel directly
- **D)** Must use the same OS as the host

Q55. The software that creates and manages virtual machines is called a:

- **A)** Device driver
- **B)** Shell
- **C)** Hypervisor
- **D)** Middleware

Q56. In virtualization terminology, the OS running within a virtual machine is called the:

- **A)** Host OS
- **B)** Guest OS
- **C)** Hypervisor OS
- **D)** Base OS

Q57. Emulation differs from virtualization in that emulation:

- **A)** Creates VMs for the same CPU architecture
- **B)** Simulates computer hardware when the source CPU type is different from the target CPU type
- **C)** Is faster than virtualization
- **D)** Does not involve any software

Q58. Apple's 'Rosetta' is an example of:

- **A)** Virtualization (same architecture)
- **B)** Emulation (IBM PowerPC apps running on Intel x86)
- **C)** A hypervisor
- **D)** A guest OS

Q59. Which company created virtualization technology as a Windows application for running multiple guest copies of Windows?

- **A)** Microsoft
- **B)** Apple
- **C)** VMware
- **D)** IBM

Q60. Benefits of virtualization include all of the following EXCEPT:

- **A)** Resource Consolidation
- **B)** Isolation between VMs
- **C)** Increased hardware procurement cost
- **D)** Easy testing in different OS environments

Q61. A key part of the OS system call flow — after the kernel executes the request — is:

- **A)** The user program is terminated
- **B)** The result is returned to the user program and it resumes execution
- **C)** The kernel mode is never exited
- **D)** The process is moved to the blocked queue

Q62. In the write() system call, data is copied from the user buffer to a kernel buffer to:

- **A)** Speed up execution
- **B)** Protect the kernel from potentially malicious user data
- **C)** Save power
- **D)** Avoid paging

Q63. Which of the following is a Communication system call in UNIX?

- **A)** getpid()
- **B)** open()
- **C)** pipe()
- **D)** ioctl()

Q64. exec() system call is used after fork() to:

- **A)** Terminate the child process
- **B)** Replace the child's memory space with a new program
- **C)** Wait for the child to complete
- **D)** Create another child process

Q65. The usecs/call column in strace -c output measures:

- **A)** Total CPU time in seconds
- **B)** The number of microseconds per individual system call invocation
- **C)** Memory consumed per call
- **D)** Number of arguments per call

Section 4: Processes & PCB

Q66. Which of the following BEST defines a process?

- **A)** A compiled binary file stored on disk
- **B)** A program in execution — an active entity with state, resources, and a thread of execution
- **C)** A hardware interrupt handler
- **D)** A file descriptor in the OS file table

Q67. A program is to a process as a _____ is to a _____.

- **A)** Recipe : Meal being cooked
- **B)** Car : Road

- **C)** Disk : RAM
- **D)** Thread : Kernel

Q68. The Program Counter (PC) in a process stores:

- **A)** The number of instructions executed
- **B)** The address of the NEXT instruction to be executed
- **C)** The result of the last arithmetic operation
- **D)** The base address of the process in memory

Q69. Which component of a process stores dynamically allocated memory (e.g., malloc)?

- **A)** Stack
- **B)** Text segment
- **C)** Heap
- **D)** BSS segment

Q70. Local variables and function call return addresses are stored in the:

- **A)** Heap
- **B)** Data segment
- **C)** BSS segment
- **D)** Stack

Q71. Uninitialized global variables in a C program are stored in:

- **A)** Text segment
- **B)** Stack
- **C)** BSS segment
- **D)** Heap

Q72. Initialized global variables in a C program are stored in the:

- **A)** Text segment
- **B)** Data segment
- **C)** Stack
- **D)** Heap

Q73. The Text segment of a process is typically:

- **A)** Read-write and unique to each process instance
- **B)** Read-only and shared among multiple process instances of the same program
- **C)** Part of the kernel address space
- **D)** Allocated dynamically at runtime

Q74. Which direction does the stack grow in memory?

- **A)** Toward high addresses (upward)
- **B)** Toward low addresses (downward)
- **C)** Does not move — it is fixed size

- **D)** Alternates direction depending on CPU architecture

Q75. Which direction does the heap grow in memory?

- **A)** Toward low addresses (downward)
- **B)** Toward high addresses (upward)
- **C)** It does not grow — it is pre-allocated
- **D)** Toward the text segment

Q76. An I/O-bound process is one that:

- **A)** Generates I/O requests infrequently and uses more CPU time
- **B)** Spends more time doing I/O than computation
- **C)** Never performs any I/O operations
- **D)** Uses the CPU for 100% of its execution

Q77. A CPU-bound process:

- **A)** Makes frequent I/O requests and waits often
- **B)** Has many short CPU bursts and long I/O waits
- **C)** Generates I/O requests infrequently; uses most of its time doing computations
- **D)** Is always interactive with the user

Q78. The PCB is described as "key" because:

- **A)** It provides hardware access to the CPU
- **B)** It enables the OS to support multiple processes by storing enough information to interrupt and resume any process
- **C)** It stores the compiled binary code of every process
- **D)** It contains the OS kernel itself

Q79. Which PCB field is IMMUTABLE throughout the lifetime of a process?

- **A)** State
- **B)** Program Counter
- **C)** Process Identifier (PID)
- **D)** Priority

Q80. Context data in the PCB contains:

- **A)** The process's open file list
- **B)** The values of all CPU registers at the time the process was last preempted
- **C)** The process's accounting history
- **D)** The page table of the process

Q81. What is stored in the Dispatcher Queue?

- **A)** Pointers to the PCB structures of processes waiting for the CPU
- **B)** Actual compiled code of each process
- **C)** The process's stack data

- **D)** Device interrupt vectors

Q82. In Linux, the kernel stores per-process information in a structure called:

- **A)** proc_info
- **B)** kernel_struct
- **C)** task_struct
- **D)** process_block

Q83. When the OS creates a new process, it first:

- **A)** Allocates CPU time immediately
- **B)** Creates a PCB for the process and enters it in the Not Running / New state
- **C)** Loads the process into the Ready Queue immediately
- **D)** Assigns a file descriptor to the process

Q84. CPU Burst is defined as:

- **A)** The time a process waits for I/O to complete
- **B)** The period during which the process is actively executing instructions on the CPU
- **C)** The time from process creation to first execution
- **D)** The total execution time including I/O waits

Q85. Context switch time is considered:

- **A)** Productive CPU time
- **B)** Pure overhead — no useful work is done during the switch
- **C)** Part of the process's burst time
- **D)** A form of I/O operation

Q86. Context switching is written in assembly language because:

- **A)** It is simpler than writing in C
- **B)** It runs faster in assembly
- **C)** It directly manipulates CPU registers and must execute atomically, tightly coupled to hardware
- **D)** C compilers do not support context switch operations

Q87. Which PCB fields change when a process transitions from Running to Blocked state due to I/O?

Choose the BEST answer:

- **A)** Only the State field changes
- **B)** State, PC, Context Data, I/O Status Info, and Accounting Info all change
- **C)** Only the Program Counter changes
- **D)** No PCB fields change during this transition

Q88. A process in the Ready Queue is waiting for:

- **A)** An I/O device to complete
- **B)** A signal from another process
- **C)** CPU allocation from the dispatcher
- **D)** Memory to be allocated by the OS

Q89. The multiprogramming model gives the illusion of simultaneous execution. The REALITY is:

- **A)** Multiple CPUs are always used
- **B)** Only one process is actually active on a single CPU at any instant
- **C)** All processes share the CPU registers simultaneously
- **D)** The OS kernel runs all processes together in parallel

Q90. Which command in Linux can be used to view the process table?

- **A)** ls
 - **B)** grep
 - **C)** ps and top
 - **D)** cat /proc/kernel
-

Section 5: Process State Models

Q91. In the Two-State Process Model, the two states are:

- **A)** Ready and Blocked
- **B)** Running and Not Running
- **C)** New and Exit
- **D)** Ready and Running

Q92. The main LIMITATION of the Two-State Model is:

- **A)** It has too many states to track
- **B)** It conflates processes that are ready-to-run with processes that are blocked on I/O in a single queue
- **C)** It does not include a Running state
- **D)** It cannot model process creation

Q93. In the Three-State Model, which state represents a process that has all resources EXCEPT the CPU?

- **A)** Running
- **B)** Blocked
- **C)** Ready
- **D)** New

Q94. The transition "Running → Blocked" is triggered by:

- **A)** Time quantum expiry
- **B)** The dispatcher selecting this process

- **C)** Process initiating an I/O request or making a blocking system call
- **D)** Process completing execution

Q95. The transition "Blocked → Ready" is triggered by:

- **A)** Time quantum expiry
- **B)** The dispatcher selecting this process
- **C)** I/O completing or an event occurring
- **D)** The process calling exit()

Q96. The transition "Running → Ready" is triggered by:

- **A)** I/O request from the process
- **B)** Time slice (quantum) expiry or preemption by higher-priority process
- **C)** I/O completion
- **D)** Process termination

Q97. In the Five-State Model, the "New" state means:

- **A)** Process is running on the CPU for the first time
- **B)** PCB has been created by the OS but the resident set is NOT yet loaded into main memory
- **C)** Process has just completed execution
- **D)** Process is waiting for the first I/O operation

Q98. The transition from New to Ready occurs when:

- **A)** The process is dispatched to the CPU
- **B)** Loading of the process's resident set into main memory is completed
- **C)** The process makes its first I/O request
- **D)** The process is admitted to the batch queue

Q99. The "Exit" state in the Five-State Model serves what purpose?

- **A)** Process is waiting for CPU assignment
- **B)** OS retains the PCB temporarily for auditing and in case the same process is re-created (image available in MM)
- **C)** Process is swapped to disk
- **D)** Process is waiting for a child to terminate

Q100. The Suspend state is introduced in the OS because:

- **A)** The Ready queue has too many processes
- **B)** MM may be full of blocked processes, leaving no ready processes — CPU starves
- **C)** The CPU has too many tasks to perform
- **D)** The disk drive is too slow

Q101. When a process is "Suspended," where is its resident set stored?

- **A)** In the CPU cache
- **B)** In the kernel's page cache

- **C)** On disk (secondary storage)
- **D)** In a separate segment of main memory

Q102. When an OS chooses which blocked process to suspend, it selects:

- **A)** The process with the highest burst time
- **B)** The process with the LOWEST priority
- **C)** The most recently created process
- **D)** The process with the most open files

Q103. After a suspended process's blocking event occurs, it is brought from disk into:

- **A)** Blocked state in MM
- **B)** New state
- **C)** Directly into Ready state (bypassing Blocked state)
- **D)** Exit state

Q104. "Blocked, Suspend" means a process is:

- **A)** Ready to run but swapped out
- **B)** Waiting for an event AND swapped out of main memory
- **C)** Running on the CPU in a low-priority mode
- **D)** Terminated but PCB retained

Q105. "Ready, Suspend" means a process is:

- **A)** Waiting for I/O on disk
- **B)** Ready to run but swapped out of MM to disk; waiting to be swapped back in
- **C)** Running with reduced privileges
- **D)** In the batch queue not yet admitted

Q106. The dispatcher queue is organized as which data structure?

- **A)** Stack (LIFO)
- **B)** Tree
- **C)** First-In-First-Out (FIFO) queue
- **D)** Hash table

Q107. A process can stay in the Blocked state for:

- **A)** Exactly one time quantum
- **B)** At most one CPU burst duration
- **C)** An unlimited amount of time — depends on occurrence of an external event
- **D)** No more than 1 millisecond

Q108. In the Three-State Model, having separate Ready and Blocked queues allows the dispatcher to:

- **A)** Always pick a random process
- **B)** Immediately select an unblocked ready process without scanning all processes
- **C)** Run processes in order of PID
- **D)** Keep the blocked queue synchronized with the disk

Q109. Which transition does NOT exist in the basic Five-State Model?

- **A)** New → Ready
- **B)** Ready → Running
- **C)** Blocked → Running (directly, without going through Ready)
- **D)** Running → Blocked

Q110. In UNIX, when a parent process terminates without calling wait() on its children, those children are called:

- **A)** Zombie processes
 - **B)** Daemon processes
 - **C)** Orphan processes
 - **D)** Suspended processes
-

Section 6: Threads & IPC

Q111. A thread is also called a:

- **A)** Heavyweight process
- **B)** Lightweight process (LWP)
- **C)** Kernel object
- **D)** Device driver

Q112. Which of the following is shared among all threads of the same process?

- **A)** Stack
- **B)** Program Counter
- **C)** Register Set
- **D)** Address space (code, data, files)

Q113. Which of the following is PRIVATE to each thread (NOT shared)?

- **A)** Global variables
- **B)** Open files
- **C)** Stack and stack pointer
- **D)** Code segment

Q114. The advantage of threads over processes for related tasks is:

- **A)** Threads have separate address spaces ensuring strong isolation
- **B)** Threads are faster to create/destroy and share memory directly without IPC overhead
- **C)** Threads always run faster due to larger memory allocation
- **D)** Threads are managed entirely by the hardware

Q115. In a multithreaded word processor, which thread reads keyboard input?

- **A)** Format document thread
- **B)** Write-to-disk thread
- **C)** Keyboard input thread
- **D)** Display rendering thread

Q116. User-Level Threads (ULT) are managed by:

- **A)** The OS kernel directly
- **B)** A user-space thread library
- **C)** The CPU hardware scheduler
- **D)** The device driver layer

Q117. A key DISADVANTAGE of User-Level Threads is:

- **A)** They are slower to context switch than kernel threads
- **B)** One thread making a blocking I/O call blocks the entire process
- **C)** They require more memory than kernel threads
- **D)** They cannot be used in user applications

Q118. Kernel-Level Threads (KLT) support:

- **A)** Faster context switching than ULT
- **B)** No OS awareness of individual threads
- **C)** True parallelism on multi-core CPUs
- **D)** No blocking on I/O calls

Q119. Which thread model has a higher overhead for context switching?

- **A)** User-Level Threads
- **B)** Kernel-Level Threads
- **C)** Both have identical overhead
- **D)** Neither — threads never perform context switches

Q120. In the multithreaded web server, the Dispatcher thread's role is:

- **A)** Serve each HTTP request directly
- **B)** Read web pages from disk
- **C)** Fetch incoming requests and hand them off to worker threads
- **D)** Manage the web page cache

Q121. Context switching between threads of the same process is cheaper than context switching between different processes because:

- **A)** Threads use smaller stacks
- **B)** No address space switch is needed — threads share the same virtual address space
- **C)** Threads have no registers to save
- **D)** The kernel is not involved in thread scheduling

Q122. A process with exactly one thread is equivalent to:

- **A)** A multithreaded process
- **B)** A classical (traditional) process
- **C)** A daemon process
- **D)** A zombie process

Q123. Which of the following is a reason for using IPC?

- **A)** To compile programs faster
- **B)** To allow cooperating processes to exchange data safely and efficiently
- **C)** To replace the OS scheduler
- **D)** To eliminate the need for a file system

Q124. The two models of IPC are:

- **A)** Synchronous and Asynchronous
- **B)** Shared Memory and Message Passing
- **C)** Preemptive and Non-preemptive
- **D)** Blocking and Non-blocking

Q125. Which IPC model is FASTER for large data exchange?

- **A)** Message Passing
- **B)** Shared Memory
- **C)** Both are equally fast
- **D)** Socket-based IPC

Q126. In the Shared Memory IPC model:

- **A)** All data exchange goes through the OS kernel for every operation
- **B)** Processes map a common memory region and read/write directly after initial setup
- **C)** Processes can only exchange data at fixed intervals
- **D)** The OS handles all synchronization automatically

Q127. Which UNIX system call creates a shared memory segment?

- **A)** malloc()
- **B)** fork()
- **C)** shmget()
- **D)** pipe()

Q128. pipe() in UNIX is an example of which IPC model?

- **A)** Shared Memory
- **B)** Message Passing
- **C)** Direct hardware access
- **D)** Virtual memory sharing

Q129. How many threads can exist inside one process?

- **A)** Exactly one
- **B)** Exactly two
- **C)** Multiple threads can exist in one process
- **D)** Only the number of CPU cores available

Q130. The Thread Control Block (TCB) stores:

- **A)** The process's shared global variables
 - **B)** Thread-specific state: Thread ID, PC, registers, stack pointer, state
 - **C)** The OS kernel data structures
 - **D)** The parent process's PCB
-

Section 7: Schedulers & Scheduling Criteria

Q131. The Short-Term Scheduler is also called the:

- **A)** Job Scheduler
- **B)** Swapper
- **C)** CPU Scheduler / Dispatcher
- **D)** Long-Term Scheduler

Q132. The Long-Term Scheduler is also known as the:

- **A)** CPU Scheduler
- **B)** Job Scheduler
- **C)** Dispatcher
- **D)** Medium-Term Scheduler

Q133. The Medium-Term Scheduler performs:

- **A)** Selecting which process gets the CPU from the Ready queue
- **B)** Admitting new jobs from the batch queue into the system
- **C)** Swapping processes in and out of main memory
- **D)** Handling hardware interrupts

Q134. Which scheduler controls the "degree of multiprogramming"?

- **A)** Short-Term Scheduler
- **B)** Medium-Term Scheduler
- **C)** Long-Term Scheduler
- **D)** I/O Scheduler

Q135. The Short-Term Scheduler runs how frequently?

- **A)** Once per hour
- **B)** Once when the system boots
- **C)** Very frequently — milliseconds apart — for every process switch
- **D)** Only when a process terminates

Q136. The Long-Term Scheduler runs:

- **A)** Every millisecond
- **B)** After every context switch
- **C)** Infrequently — invoked when a completed job departs the system
- **D)** Only during system boot

Q137. The primary objective of the Long-Term Scheduler is to:

- **A)** Maximize CPU utilization for a single process
- **B)** Maintain a balanced mix of I/O-bound and CPU-bound processes
- **C)** Minimize disk I/O operations
- **D)** Select the shortest job from the ready queue

Q138. A process is swapped OUT by the Medium-Term Scheduler when:

- **A)** It requests too many files
- **B)** MM is full and the process is blocked — swapping frees MM for other processes
- **C)** The process has completed execution
- **D)** The process's priority is too high

Q139. The Short-Term Scheduler is invoked by all events EXCEPT:

- **A)** Time slice (quantum) completion
- **B)** I/O completion
- **C)** OS system call completion
- **D)** A new batch job arriving on disk

Q140. Suspended queues are maintained in:

- **A)** The CPU cache
- **B)** The hard disk directly as raw data
- **C)** OS data structures in main memory (only the resident-set of swapped processes is on HDD)
- **D)** GPU memory

Q141. The scheduling criterion "Throughput" means:

- **A)** Total time taken for one process to complete
- **B)** Percentage of time the CPU is busy
- **C)** Number of processes completing execution per unit time
- **D)** Time until the first response from an interactive process

Q142. "Turnaround Time" is calculated as:

- **A)** Completion Time – Burst Time
- **B)** Completion Time – Arrival Time
- **C)** Burst Time + Priority
- **D)** Waiting Time × CPU Utilization

Q143. "Waiting Time" is calculated as:

- **A)** Completion Time – Arrival Time
- **B)** Turnaround Time – Burst Time
- **C)** Burst Time + Arrival Time
- **D)** Completion Time × Throughput

Q144. "Response Time" is the time from:

- **A)** Process creation to process termination
- **B)** When a job request is submitted until the FIRST response is produced
- **C)** When a process enters the ready queue until it starts executing for the last time
- **D)** First execution to final completion

Q145. Which scheduling criterion should be MINIMIZED?

- **A)** CPU Utilization
- **B)** Throughput
- **C)** Waiting Time
- **D)** All of the above should be minimized

Q146. Which scheduling criterion should be MAXIMIZED?

- **A)** Turnaround Time
- **B)** Waiting Time
- **C)** CPU Utilization
- **D)** Response Time

Q147. Why is the waiting time a process spends in the BLOCKED queue NOT counted for scheduling performance?

- **A)** Because blocked time is always very short
- **B)** Because the scheduler has no control over it — the process is waiting for its own I/O request
- **C)** Because blocked processes are not tracked by the OS
- **D)** Because blocked time counts as CPU time

Q148. Scheduling algorithms typically optimize:

- **A)** Exact individual process values
- **B)** Average values across all processes
- **C)** Only the highest-priority process's metrics
- **D)** Only the most recently arrived process's metrics

Q149. CPU utilization when context switch time is negligible and no CPU idle gaps exist equals:

- **A)** 50%
- **B)** 75%
- **C)** 100%
- **D)** Depends on the number of processes

Q150. Preemptive scheduling allows:

- **A)** A running process to voluntarily release the CPU whenever it wishes
 - **B)** The OS to forcibly remove a running process from the CPU (quantum expiry or higher priority arrival)
 - **C)** No interruption of running processes under any circumstances
 - **D)** Only hardware interrupts to stop a running process
-

Section 8: Scheduling Algorithms — Conceptual

Q151. FCFS scheduling is equivalent to a Round-Robin scheduler with quantum = :

- **A)** 0
- **B)** 1
- **C)** ∞ (infinity)
- **D)** The average burst time

Q152. FCFS is characterized as:

- **A)** A preemptive, fair scheduler
- **B)** A non-preemptive scheduler that favors earlier-arriving processes
- **C)** An optimal scheduler minimizing average waiting time
- **D)** A real-time scheduler with deadline guarantees

Q153. The "Convoy Effect" in FCFS occurs when:

- **A)** Short processes overtake long processes
- **B)** One long CPU-bound process holds the CPU while many short processes wait behind it
- **C)** The CPU is shared equally among all processes
- **D)** Multiple processes simultaneously access the I/O device

Q154. Round Robin scheduling is BEST suited for:

- **A)** Batch processing systems
- **B)** Time-sharing / interactive environments
- **C)** Real-time hard deadline systems
- **D)** Single-process environments

Q155. In Round Robin, the Quantum Time (q) is also called:

- **A)** Context switch time
- **B)** Time slice
- **C)** Burst time
- **D)** Arrival time

Q156. In Round Robin, context switching time should be:

- **A)** Greater than the quantum time
- **B)** Equal to the quantum time
- **C)** Less than 1% of the quantum time
- **D)** Exactly 10% of the quantum time

Q157. If n processes run under Round Robin, each process perceives a CPU running at:

- **A)** $n \times$ actual speed
- **B)** Actual speed
- **C)** $1/n \times$ actual speed
- **D)** $2/n \times$ actual speed

Q158. If the Round Robin quantum is made very large (approaches infinity), it degenerates to:

- **A)** SJF
- **B)** SRTF
- **C)** FCFS
- **D)** Priority Scheduling

Q159. Reducing the time slice in Round Robin:

- **A)** Decreases average turnaround time
- **B)** Increases responsiveness but increases average turnaround and waiting time due to more context switches
- **C)** Has no effect on system performance
- **D)** Eliminates context switching overhead

Q160. In Round Robin, when two processes arrive at the same time, which enters the Ready queue FIRST?

- **A)** The process with the shorter burst time
- **B)** The process with the higher priority
- **C)** The process with the lower process ID
- **D)** The last process to arrive (most recent)

Q161. SJF (non-preemptive) selects the process with the:

- **A)** Longest burst time
- **B)** Earliest arrival time
- **C)** Smallest burst time

- **D)** Highest priority value

Q162. SJF is provably optimal in terms of:

- **A)** Minimizing average response time
- **B)** Minimizing average waiting time among non-preemptive algorithms
- **C)** Maximizing throughput
- **D)** Minimizing average turnaround time

Q163. The main PRACTICAL limitation of SJF scheduling is:

- **A)** It requires preemption hardware
- **B)** The burst time of a process is generally not known in advance
- **C)** It can only handle two processes at a time
- **D)** It does not support multi-core systems

Q164. In SJF scheduling, when two processes have the same burst time, the tie is broken by:

- **A)** Highest priority
- **B)** FCFS — the process that arrived earlier runs first
- **C)** Random selection
- **D)** Alphabetical order of process name

Q165. SRTF is the preemptive version of:

- **A)** FCFS
- **B)** Round Robin
- **C)** SJF
- **D)** Priority Scheduling

Q166. In SRTF, a running process is preempted when:

- **A)** Its quantum expires
- **B)** A new process arrives with a shorter REMAINING burst time than the current process's remaining time
- **C)** Any new process arrives
- **D)** The ready queue has more than 5 processes

Q167. SRTF provides:

- **A)** The worst average waiting time of all algorithms
- **B)** The globally optimal minimum average waiting time
- **C)** The same average waiting time as FCFS
- **D)** The highest throughput regardless of burst times

Q168. Round Robin guarantees:

- **A)** Minimum average waiting time
- **B)** No starvation — every ready process eventually gets CPU time
- **C)** Maximum CPU utilization
- **D)** Optimal turnaround time for all processes

Q169. Which scheduling algorithm CAN lead to starvation of long processes?

- **A)** FCFS
- **B)** Round Robin
- **C)** SJF
- **D)** Both A and B

Q170. The key difference between SJF and SRTF is:

- **A)** SJF runs shorter jobs first; SRTF runs longer jobs first
- **B)** SJF is non-preemptive; SRTF is preemptive
- **C)** SJF is preemptive; SRTF is non-preemptive
- **D)** They are identical algorithms with different names

Section 9: Scheduling Algorithms — Numerical

Q171. Processes P1 (burst=8) and P2 (burst=12) both arrive at $t=0$. Under non-preemptive scheduling, the Average Waiting Time is:

- **A)** 2
- **B)** 4
- **C)** 6
- **D)** 8

Q172. Using the same processes as Q171, Average Turnaround Time is:

- **A)** 10
- **B)** 12
- **C)** 14
- **D)** 16

Q173. P1 (burst=8) arrives at $t=0$; P2 (burst=12) arrives at $t=4$. Non-preemptive. Average Turnaround Time:

- **A)** 10
- **B)** 12
- **C)** 14
- **D)** 16

Q174. For the same processes as Q173, Average Waiting Time is:

- **A)** 1
- **B)** 2
- **C)** 3
- **D)** 4

Q175. P1 (burst=8, arrive=0), P2 (burst=12, arrive=4), P3 (burst=4, arrive=22). Non-preemptive. CPU Utilization is approximately:

- **A)** 100%
- **B)** 96%
- **C)** 92.3%

- **D)** 88%

Q176. Same processes as Q175. Throughput is:

- **A)** 3/22
- **B)** 3/26
- **C)** 2/20
- **D)** 3/24

Q177. P1 (burst=8) and P2 (burst=12) both arrive at $t=0$. Round Robin with quantum=4. The Gantt chart is:

- **A)** P1 P2 P2 P2
- **B)** P1 P2 P1 P2 P2
- **C)** P2 P1 P2 P1 P2
- **D)** P1 P1 P2 P2 P2

Q178. Using the same processes as Q177 (RR, $q=4$), Average Waiting Time is:

- **A)** 4
- **B)** 5
- **C)** 6
- **D)** 8

Q179. Using the same processes as Q177 (RR, $q=4$), Average Turnaround Time is:

- **A)** 14
- **B)** 16
- **C)** 18
- **D)** 20

Q180. P1 (burst=8) and P2 (burst=12) both arrive at $t=0$. RR with quantum=3. Average Turnaround Time is:

- **A)** 16
- **B)** 17
- **C)** 18
- **D)** 20

Q181. Same processes as Q180 (RR, $q=3$). Average Waiting Time is:

- **A)** 5
- **B)** 6
- **C)** 7
- **D)** 8

Q182. For the SJF non-preemptive problem with P1(3,1), P2(1,4), P3(4,2), P4(0,6), P5(2,3) [format: arrival, burst], which process runs FIRST?

- **A)** P1
- **B)** P2

- **C)** P3
- **D)** P4

Q183. In the same SJF problem, after P4 completes at $t=6$, which process runs next?

- **A)** P2 (burst=4)
- **B)** P3 (burst=2)
- **C)** P1 (burst=1)
- **D)** P5 (burst=3)

Q184. The complete SJF (non-preemptive) execution order for P1(3,1), P2(1,4), P3(4,2), P4(0,6), P5(2,3) is:

- **A)** P4 → P1 → P3 → P5 → P2
- **B)** P4 → P2 → P5 → P3 → P1
- **C)** P2 → P5 → P1 → P3 → P4
- **D)** P1 → P2 → P3 → P4 → P5

Q185. In the SJF problem from Q184, P2's Turnaround Time is:

- **A)** 4
- **B)** 8
- **C)** 12
- **D)** 15

Q186. In the SJF problem from Q184, P1's Waiting Time ($WT = TAT - BT$) is:

- **A)** 0
- **B)** 3
- **C)** 4
- **D)** 6

Q187. In the SRTF problem with P1(3,1), P2(1,4), P3(4,2), P4(0,6), P5(2,3) [arrival,burst]: At $t=1$, P2 arrives with burst=4. Currently P4 has 5 units remaining. What happens?

- **A)** P4 continues running ($4 > 5$ is false, P4 stays)
- **B)** P2 preempts P4 because P2's remaining time (4) < P4's remaining time (5)
- **C)** P4 continues because it arrived first
- **D)** Both P2 and P4 share the CPU

Q188. In the same SRTF problem, at $t=3$, P1 arrives with burst=1. Currently P2 has 2 units remaining. What happens?

- **A)** P2 continues (it arrived before P1)
- **B)** P1 preempts P2 because P1's remaining time (1) < P2's remaining time (2)
- **C)** P1 enters the queue but P2 is not preempted
- **D)** P1 is rejected because P2 is already running

Q189. In the SRTF problem from Q187, P1 completes at time:

- **A) 3**
- **B) 4**
- **C) 5**
- **D) 6**

Q190. In the SRTF problem, which process has the SHORTEST waiting time and why?

- **A) P4, because it arrived first**
- **B) P1, because it has the shortest burst time (1 unit) → runs immediately when it arrives**
- **C) P2, because it arrived at $t=1$**
- **D) P3, because it arrived later**

Q191. P1(0,5), P2(1,3), P3(2,8), P4(3,6). FCFS. Turnaround Time for P2 is:

- **A) 3**
- **B) 7**
- **C) 5**
- **D) 4**

Q192. Using FCFS with P1(0,5), P2(1,3), P3(2,8), P4(3,6), Average Waiting Time is:

- **A) 4.25**
- **B) 5.75**
- **C) 6.00**
- **D) 7.00**

Q193. Process P1 has arrival=0, burst=6. Process P2 has arrival=2, burst=4. Under SJF (non-preemptive), which runs first?

- **A) P2, because it has a shorter burst**
- **B) P1, because it arrived first and SJF is non-preemptive (only P1 is available at $t=0$)**
- **C) They run concurrently**
- **D) P2 preempts P1 at $t=2$**

Q194. Same processes as Q193. Under SRTF, at $t=2$ when P2 arrives, does P2 preempt P1?

- **A) No — P1's remaining time (4) = P2's burst (4), tie → FCFS → P1 stays**
- **B) Yes — P2 preempts P1 immediately**
- **C) No — SRTF never preempts**
- **D) Yes — any new arrival always preempts**

Q195. In Round Robin ($q=2$) with process A(0,4), B(0,12), C(2,6), at $t=2$ a new process C arrives while A is running. When A's quantum ends at $t=2$, the Ready Queue is built as:

- **A)** C enters before the preempted A (newly arrived gets priority over preempted)
- **B)** A enters before C (preempted process has priority)
- **C)** C and A enter simultaneously with no order
- **D)** C waits until $t=4$ to enter the queue

Q196. Two processes: P1 burst=8, P2 burst=12, both arrive at $t=0$. Under RR ($q=4$), at what time does P1 finish?

- **A)** 8
- **B)** 12
- **C)** 16
- **D)** 20

Q197. Same as Q196. At what time does P2 finish?

- **A)** 12
- **B)** 16
- **C)** 20
- **D)** 24

Q198. For SJF, the ready queue is sorted by:

- **A)** Arrival time (ascending)
- **B)** Process ID (ascending)
- **C)** Burst time (ascending — smallest burst at head)
- **D)** Priority (descending)

Q199. P1(0,4), P2(0,3), P3(0,2). SJF non-preemptive. Execution order (all arrive at $t=0$):

- **A)** P1 → P2 → P3
- **B)** P3 → P2 → P1
- **C)** P2 → P3 → P1
- **D)** P1 → P3 → P2

Q200. Same as Q199. Average Waiting Time is:

- **A)** 1.67
- **B)** 2.0
- **C)** 3.0
- **D)** 3.67

PART B: FILL IN THE BLANKS

Write the correct term or value in each blank.

Section B1: Unit I — Definitions & Types

- B1.** An OS acts as an intermediary between the _____ and the computer hardware.
- B2.** The one program running at all times on the computer, the core of the OS, is called the _____.
- B3.** The OS performs no useful function by itself — it provides an _____ within which other programs can do useful work.
- B4.** Programs associated with the OS but not part of the kernel are called _____ programs.
- B5.** OS/390 is an example of a _____ Operating System.
- B6.** A Mainframe OS typically offers three services: batch, _____, and timesharing.
- B7.** Google Android and Apple iOS are examples of _____ Computer Operating Systems.
- B8.** Embedded OS software is stored in _____ so no user-installed applications are possible.
- B9.** TinyOS is an example of an OS designed for _____ networks.
- B10.** A _____ Real-Time OS must provide absolute guarantees that actions occur by a certain time; missing a deadline is catastrophic.
- B11.** Smartphones are classified as _____ real-time systems.
- B12.** Smart card OSes typically hold an interpreter for the _____ in their ROM.
- B13.** Other names for a Multiprocessor OS include parallel computers, multi-computers, and _____.

B14. Solaris, FreeBSD, and Windows Server are examples of _____ Operating Systems.

B15. TinyOS is event-driven because sensor nodes have limited _____ and battery lifetime constraints.

Section B2: OS Services, Structure, Booting

B16. The required contents of a program that must reside in main memory for it to execute (Code + Data + Stack) are collectively called the _____ of the program.

B17. The _____ abstracts access to different hardware peripherals with a common set of APIs.

B18. A device _____ understands a specific device controller and provides the rest of the OS with a uniform interface.

B19. During cold boot, the first program that executes is the _____ program.

B20. After the bootstrap program, the OS _____ is located and loaded into main memory.

B21. The OS then starts executing the first process, typically called "_____" in UNIX systems.

B22. Hardware interrupts are sent to the CPU via the _____.

B23. A software interrupt triggered by executing a special operation is called a _____ (or monitor call).

B24. _____ booting refers to restarting the computer using the OS restart function (without a full power cycle).

B25. UNIX and Linux OS have a _____ structure where all OS components run in a single kernel executable.

B26. The OS collects usage statistics and monitors performance parameters as part of the _____ service.

B27. Access control such as username/password to login is part of the _____ service of the OS.

B28. Step 3 of the interrupt handling sequence is: "Assembly language routine _____ registers."

B29. Step 7 of the interrupt handling sequence is: "Scheduler _____ a process to run next."

B30. The _____ layer of the operating system contains the most frequently used functions and is always resident in main memory.

Section B3: System Calls & Virtualization

B31. A system call is a request from a _____ process to the kernel to perform a specific privileged task.

B32. When a system call is invoked, the CPU switches from _____ Mode to _____ Mode.

B33. The mechanism that causes the CPU mode switch during a system call is called a _____.

B34. File descriptor 0 corresponds to _____ (standard input).

B35. File descriptor 1 corresponds to _____ (standard output).

B36. File descriptor 2 corresponds to _____ (standard error).

B37. fork(), exec(), and exit() are examples of _____ Control system calls.

B38. open(), read(), write(), and close() are examples of _____ Management system calls.

B39. getpid(), alarm(), and sleep() are examples of _____ Maintenance system calls.

B40. pipe(), msgsnd(), and msgrcv() are examples of _____ system calls.

B41. In write(1, message, sizeof(message)), the value 1 is the _____ for standard output.

B42. The Linux command strace -c ls displays a _____ of system calls with timing.

- B43.** usecs/call in strace output measures the number of _____ per individual system call.
- B44.** Applications are OS-specific because each OS provides a unique set of _____.
- B45.** Writing in an interpreted language like _____ or Ruby is one solution for making apps portable.
- B46.** _____ technology abstracts hardware of a single computer into several virtual execution environments.
- B47.** The OS running on the physical hardware is called the _____ OS.
- B48.** The OS running within a virtual machine is called the _____ OS.
- B49.** The software that creates and manages virtual machines is called a _____.
- B50.** Apple's '_____' was an emulation facility for running IBM PowerPC apps on Intel x86.

Section B4: Processes, PCB, Memory Layout

- B51.** A process is a program in _____.
- B52.** The _____ Counter stores the address of the next instruction to be executed.
- B53.** Dynamic memory allocated using malloc() is stored in the _____ segment.
- B54.** Local variables and function call frames are stored in the _____ segment.
- B55.** Uninitialized global variables (zeroed at startup) are stored in the _____ segment.
- B56.** Initialized global variables are stored in the _____ segment.
- B57.** Compiled machine code and string literals are stored in the _____ segment.
- B58.** The stack grows toward _____ addresses in memory.
- B59.** The heap grows toward _____ addresses in memory.

- B60.** A _____ overflow occurs when the stack grows beyond its allocated limit.
- B61.** A _____ leak occurs when dynamically allocated memory is no longer needed but is never freed.
- B62.** Each process has its own _____ Address Space — an illusion of exclusive memory access.
- B63.** A process that spends more time waiting for I/O than computing is called _____ bound.
- B64.** A process that generates I/O requests infrequently and uses most time computing is called _____ bound.
- B65.** The period where a process actively executes instructions on the CPU is called a _____ burst.
- B66.** The period where a process waits for I/O completion is called an _____ burst.
- B67.** PCB stands for _____.
- B68.** The PCB field that is IMMUTABLE throughout the lifetime of a process is the _____.
- B69.** Context data in the PCB stores the values of all _____ at the time the process was last preempted.
- B70.** Context switching is written in _____ language because it directly manipulates CPU registers.
- B71.** In Linux, the process table is viewed using the _____ command.
- B72.** The Linux kernel stores per-process information in a structure called _____.
- B73.** Context switch time is considered pure _____ because no useful work is done during it.
- B74.** In the Dispatcher Queue, each entry is a pointer to the _____ of a process.
- B75.** The state field in the PCB is implemented in C as an _____.

Section B5: Process States, Scheduling

B76. In the Two-State Model, the two states are _____ and _____.

B77. In the Three-State Model, the three states are _____, _____, and _____.

B78. In the Five-State Model, the state where the PCB exists but the resident set is not yet in MM is the _____ state.

B79. The transition from Running → Blocked is triggered by the process making an _____ request.

B80. The transition from Blocked → Ready is triggered by _____ completing or an event occurring.

B81. The transition from Running → Ready is triggered by _____ expiry or preemption.

B82. In the Five-State Model, when a process completes execution it transitions to the _____ state.

B83. A process in _____ state has been terminated but its PCB has not been removed (parent hasn't called wait()).

B84. A process whose parent terminated without calling wait() is called an _____ process.

B85. In the Suspend state model, the OS chooses the process with the _____ priority to suspend.

B86. When a suspended process's blocking event occurs, it is brought from disk directly into _____ state.

B87. Suspended queues are maintained as OS data structures in _____ memory.

B88. The _____ state model is the simplest but insufficient because it conflates ready and blocked processes.

B89. "Ready, Suspend" means the process is ready to run but has been swapped out to _____.

B90. The Dispatcher Queue is a _____ data structure (FIFO).

Section B6: Threads, IPC, Schedulers, Algorithms

B91. A thread is also called a _____ Process.

B92. Threads of the same process share: address space, code, data, and files — but each thread has its own _____ and stack.

B93. User-Level Threads are managed by a user-space _____ library.

B94. Kernel-Level Threads support true _____ on multi-core CPUs.

B95. One blocking I/O call in a User-Level Thread blocks the _____ process.

B96. A process with exactly one thread is equivalent to a _____ process.

B97. The two models of IPC are _____ Memory and _____ Passing.

B98. For large data exchange between processes, _____ Memory IPC is faster.

B99. The UNIX system call `shmget()` creates a _____ memory segment.

B100. The Short-Term Scheduler is also called the CPU Scheduler or _____.

B101. The Long-Term Scheduler is also called the _____ Scheduler.

B102. The Medium-Term Scheduler is also called the _____.

B103. The Long-Term Scheduler controls the _____ of multiprogramming.

B104. Turnaround Time = _____ Time – _____ Time.

B105. Waiting Time = Turnaround Time – _____ Time.

B106. CPU Utilization = (Total CPU Busy Time / Total Time) × _____ %.

B107. FCFS is equivalent to RR with quantum = _____.

B108. In Round Robin, context switch time must be less than _____ % of the quantum.

B109. In RR with n processes, each process perceives a CPU running at _____ times the actual speed.

B110. SJF provides the _____ average waiting time among all non-preemptive scheduling algorithms.

B111. The preemptive version of SJF is called _____.

B112. In SRTF, preemption occurs when a new arrival has a _____ remaining burst time than the currently running process.

B113. SRTF provides the globally _____ average waiting time.

B114. FCFS can lead to the _____ effect where short processes starve behind a long process.

B115. Round Robin guarantees no _____ because every ready process eventually gets CPU time.

PART C: TRUE OR FALSE

Write T (True) or F (False) for each statement.

C1. The OS kernel is always resident in main memory during system operation. ____

C2. Application programs are part of the OS kernel. ____

C3. OS/390 is an example of a Server Operating System. ____

C4. Embedded operating systems typically run in ROM and do not accept user-installed software. ____

C5. TinyOS is event-driven and designed for sensor-node networks. ____

C6. A Hard RTOS permits missing deadlines occasionally without catastrophic consequences. ____

C7. Smart card OSes run on powerful hardware with large amounts of RAM. ____

C8. The bootstrap program initializes the CPU registers and loads the kernel into memory. ____

- C9.** Warm booting involves a complete power cycle (fully switching the computer off). ____
- C10.** Hardware interrupts are triggered by software executing a special instruction. ____
- C11.** A system call causes the CPU to switch from User Mode to Kernel Mode. ____
- C12.** File descriptor 0 is for standard output (stdout). ____
- C13.** fork() creates a new process that is an exact copy of the parent. ____
- C14.** exec() creates a new child process without modifying memory. ____
- C15.** strace -c ls traces each system call line-by-line with arguments. ____
- C16.** Virtualization requires the guest OS to use the same CPU architecture as the host. ____
- C17.** A hypervisor is the software that creates and manages virtual machines. ____
- C18.** Emulation is typically used when the source and target CPU types are the same. ____
- C19.** The guest OS runs within a virtual machine; the host OS runs on physical hardware. ____
- C20.** VMware originally created virtualization as a hardware appliance. ____
- C21.** A process is a passive entity; a program is an active entity. ____
- C22.** The Text segment of a process is typically read-only and shared across multiple instances. ____
- C23.** The heap grows toward lower (decreasing) memory addresses. ____
- C24.** The stack grows toward lower (decreasing) memory addresses. ____
- C25.** BSS segment contains initialized global and static variables. ____
- C26.** A memory leak occurs when dynamically allocated memory is never freed. ____
- C27.** The Process Identifier (PID) changes every time the process transitions between states. ____

- C28.** Context data in the PCB stores the values of all CPU registers at the time of last preemption. ____
- C29.** Context switching is written in assembly language because it directly manipulates CPU registers. ____
- C30.** A CPU-bound process has many short CPU bursts and long I/O waits. ____
- C31.** In the Two-State Model, the two states are Running and Not Running. ____
- C32.** The Three-State Model adds New and Exit states to the Two-State Model. ____
- C33.** A process in the Blocked state can run if given the CPU. ____
- C34.** The transition from Blocked → Ready occurs when an I/O operation completes. ____
- C35.** In the Five-State Model, the New state means the PCB has been created but the resident set is not yet loaded into MM. ____
- C36.** When MM is full of blocked processes, the CPU may starve for lack of ready processes. ____
- C37.** When a suspended process's blocking event occurs, it goes directly from disk to Ready state. ____
- C38.** The OS always chooses the highest-priority process to suspend. ____
- C39.** A zombie process has terminated but its PCB has not been removed. ____
- C40.** In UNIX, orphan processes are re-parented to the init process (PID 1). ____
- C41.** Threads of the same process share the stack but NOT the address space. ____
- C42.** User-Level Threads have slower context switching than Kernel-Level Threads. ____
- C43.** Kernel-Level Threads support true parallelism on multi-core CPUs. ____
- C44.** Message Passing IPC is faster than Shared Memory for large data. ____
- C45.** The Short-Term Scheduler runs every millisecond or so (very frequently). ____
- C46.** The Long-Term Scheduler controls the degree of multiprogramming. ____
- C47.** Suspended queues are stored entirely on the hard disk. ____

- C48.** Waiting time in the Blocked queue is counted when measuring scheduler performance. ____
- C49.** Throughput should be maximized; Waiting Time should be minimized. ____
- C50.** FCFS scheduling is equivalent to Round Robin with quantum = ∞ . ____
- C51.** Round Robin can lead to starvation of any process. ____
- C52.** In Round Robin with n processes, each process perceives a CPU speed of $1/n$ of actual speed. ____
- C53.** SJF provides the minimum average waiting time among non-preemptive algorithms. ____
- C54.** The main practical limitation of SJF is that burst times are generally not known in advance. ____
- C55.** SRTF is the non-preemptive version of SJF. ____
- C56.** In SRTF, a new process always preempts the currently running process. ____
- C57.** SRTF provides the globally optimal minimum average waiting time. ____
- C58.** In FCFS, the convoy effect occurs when many long processes wait behind one short process. ____
- C59.** Reducing the quantum in RR improves responsiveness but increases average turnaround time. ____
- C60.** In SJF with a tie (equal burst times), the process with the earlier arrival time runs first. ____
-

PART D: ONE-WORD / ONE-LINE ANSWERS

Answer each question in one word, one number, or one short sentence.

- D1.** What is the name for the core part of the OS that is always in memory?
- D2.** What type of OS does TinyOS represent?
- D3.** What type of RTOS is a missile tracking system?
- D4.** What type of RTOS is a smartphone?

- D5.** Name one example of a Server Operating System.
- D6.** What property makes embedded OS design simpler than general-purpose OS design?
- D7.** What does a bootstrap program do when the computer is powered on?
- D8.** What type of booting uses the OS restart function without a full power cycle?
- D9.** What is the name of the signal that a hardware device sends to the CPU to request attention?
- D10.** What is a software interrupt caused by executing a system call instruction called?
- D11.** What is the term for the CPU mode in which the kernel runs with full hardware access?
- D12.** What is file descriptor number 1?
- D13.** What Linux command traces system calls made by a program?
- D14.** What UNIX system call creates a new process as a copy of the parent?
- D15.** What UNIX system call replaces a process's memory space with a new program?
- D16.** What is the software layer that creates and manages virtual machines?
- D17.** What is the OS running inside a virtual machine called?
- D18.** Name Apple's emulation technology that allowed IBM PowerPC apps to run on Intel x86.
- D19.** Give one benefit of virtualization.
- D20.** What is the difference between a program and a process in one sentence?
- D21.** What segment stores compiled machine code?
- D22.** What segment stores uninitialized global variables (zeroed at startup)?
- D23.** In which direction does the stack grow?
- D24.** In which direction does the heap grow?

- D25.** What happens when the stack grows beyond its allocated limit?
- D26.** What happens when dynamically allocated memory is never freed?
- D27.** What is the full form of PCB?
- D28.** Which PCB field stores the address of the next instruction to execute?
- D29.** Which PCB field is immutable throughout a process's lifetime?
- D30.** In what language is context switching written and why?
- D31.** What are the two states in the Two-State Process Model?
- D32.** What are the three states in the Three-State Model?
- D33.** What triggers the Running → Blocked transition?
- D34.** What triggers the Blocked → Ready transition?
- D35.** What is the state of a process whose PCB exists but whose resident set is not in MM?
- D36.** Why is the Suspend state needed in process state models?
- D37.** When a suspended process's event occurs, which state does it move to?
- D38.** What is a process called if it has terminated but its PCB has not been removed?
- D39.** What is a process called if its parent has terminated without calling wait()?
- D40.** What does UNIX call the process hierarchy formed by parent-child relationships?
- D41.** What is a thread also called (two words)?
- D42.** Name THREE items that threads of the same process share.
- D43.** Name THREE items that are PRIVATE to each thread.
- D44.** What is the key disadvantage of User-Level Threads regarding I/O?
- D45.** What type of thread supports true parallelism on multi-core CPUs?
- D46.** Name the two models of Interprocess Communication.

- D47.** Which IPC model is faster for large data exchange?
- D48.** What is the Long-Term Scheduler also called?
- D49.** What is the Short-Term Scheduler also called?
- D50.** What is the Medium-Term Scheduler also called?
- D51.** Which scheduler controls the degree of multiprogramming?
- D52.** Write the formula for Turnaround Time.
- D53.** Write the formula for Waiting Time.
- D54.** What is the FCFS scheduling algorithm equivalent to in terms of Round Robin?
- D55.** What is the quantum time also called?
- D56.** What does SRTF stand for?
- D57.** Which scheduling algorithm gives globally optimal minimum average waiting time?
- D58.** In which scheduling algorithm does the Convoy Effect occur?
- D59.** In Round Robin, what does each of n processes perceive about CPU speed?
- D60.** What is the preemptive version of SJF called?

MASTER ANSWER KEY MCQ Answers

Q	Ans	Q	Ans	Q	Ans	Q	Ans
Q1	B	Q26	C	Q51	B	Q76	B
Q2	C	Q27	B	Q52	B	Q77	C
Q3	D	Q28	B	Q53	C	Q78	B
Q4	B	Q29	B	Q54	B	Q79	C
Q5	B	Q30	C	Q55	C	Q80	B
Q6	C	Q31	B	Q56	B	Q81	A
Q7	B	Q32	C	Q57	B	Q82	C
Q8	C	Q33	B	Q58	B	Q83	B
Q9	B	Q34	C	Q59	C	Q84	B
Q10	B	Q35	D	Q60	C	Q85	B
Q11	B	Q36	B	Q61	B	Q86	C
Q12	C	Q37	C	Q62	B	Q87	B
Q13	C	Q38	B	Q63	C	Q88	C
Q14	D	Q39	B	Q64	B	Q89	B
Q15	C	Q40	C	Q65	B	Q90	C
Q16	B	Q41	B	Q66	B	Q91	B
Q17	D	Q42	B	Q67	A	Q92	B
Q18	C	Q43	B	Q68	B	Q93	C
Q19	B	Q44	A	Q69	C	Q94	C
Q20	B	Q45	B	Q70	D	Q95	C
Q21	B	Q46	C	Q71	C	Q96	B
Q22	B	Q47	B	Q72	B	Q97	B
Q23	B	Q48	C	Q73	B	Q98	B

Q24	B	Q49	C	Q74	B	Q99	B
Q25	B	Q50	B	Q75	B	Q100	B

Q	Ans	Q	Ans	Q	Ans	Q	Ans
Q101	C	Q126	B	Q151	C	Q176	B
Q102	B	Q127	C	Q152	B	Q177	B
Q103	C	Q128	B	Q153	B	Q178	C
Q104	B	Q129	C	Q154	B	Q179	B
Q105	B	Q130	B	Q155	B	Q180	B
Q106	C	Q131	C	Q156	C	Q181	C
Q107	C	Q132	B	Q157	C	Q182	D
Q108	B	Q133	C	Q158	C	Q183	C
Q109	C	Q134	C	Q159	B	Q184	A
Q110	C	Q135	C	Q160	C	Q185	D
Q111	B	Q136	C	Q161	C	Q186	B
Q112	D	Q137	B	Q162	B	Q187	B
Q113	C	Q138	B	Q163	B	Q188	B
Q114	B	Q139	D	Q164	B	Q189	B
Q115	C	Q140	C	Q165	C	Q190	B
Q116	B	Q141	C	Q166	B	Q191	B
Q117	B	Q142	B	Q167	B	Q192	B
Q118	C	Q143	B	Q168	B	Q193	B
Q119	B	Q144	B	Q169	C	Q194	A
Q120	C	Q145	C	Q170	B	Q195	A
Q121	B	Q146	C	Q171	B	Q196	B

Q122	B	Q147	B	Q172	C	Q197	C
Q123	B	Q148	B	Q173	B	Q198	C
Q124	B	Q149	C	Q174	B	Q199	B
Q125	B	Q150	B	Q175	C	Q200	A

Detailed Numerical Workings for Key MCQs

Q171–Q172 (Non-preemptive, P1=8, P2=12, both at t=0):

P1 finishes at t=8. P2 finishes at t=20.

TAT(P1)=8, TAT(P2)=20. WT(P1)=0, WT(P2)=8.

Avg WT = $(0+8)/2 = 4 \rightarrow$ Q171: B

Avg TAT = $(8+20)/2 = 14 \rightarrow$ Q172: C

Q173–Q174 (P1=8 at t=0; P2=12 at t=4):

P1 finishes at t=8. P2 starts at t=8, finishes at t=20.

TAT(P1)=8–0=8. TAT(P2)=20–4=16.

WT(P1)=0. WT(P2)=16–12=4.

Avg TAT = $(8+16)/2 = 12 \rightarrow$ Q173: B

Avg WT = $(0+4)/2 = 2 \rightarrow$ Q174: B

Q175–Q176 (P1=8@0, P2=12@4, P3=4@22):

P1: 0→8. P2: 8→20. Idle: 20→22. P3: 22→26. Total=26.

CPU busy = 8+12+4=24. Utilization = $24/26 = 92.3\% \rightarrow$ Q175: C

Throughput = $3/26 \rightarrow$ Q176: B

Q177–Q179 (RR, q=4, P1=8, P2=12, both at t=0):

t=0: P1 runs 4 → t=4: P2 runs 4 →
t=8: P1 runs 4 → t=12: P2 runs 4 →
t=16: P2 runs 4 → t=20

Gantt: P1|P2|P1|P2|P2

P1 completes at t=12. P2 completes at t=20.

TAT(P1)=12, TAT(P2)=20.

WT(P1)=12–8=4, WT(P2)=20–12=8.

Avg WT = (4+8)/2 = 6 → Q178: C

Avg TAT = (12+20)/2 = 16 → Q179: B

Q180–Q181 (RR, q=3, P1=8, P2=12, both at t=0):

P1|P2|P1|P2|P1|P2|P2 → P1 completes at t=14. P2 at t=20.

TAT(P1)=14, TAT(P2)=20.

WT(P1)=14–8=6, WT(P2)=20–12=8.

Avg TAT=(14+20)/2=17 → Q180: B.

Avg WT=(6+8)/2=7 → Q181: C

**Q184–Q186 (SJF non-preemptive:
P1(3,1), P2(1,4), P3(4,2), P4(0,6),
P5(2,3)):**

t=0: Only P4(6) available → P4 runs until t=6.

By t=6: P1(1), P2(4), P3(2), P5(3) all in queue → sorted:

$P1(1) < P3(2) < P5(3) < P2(4)$

t=6: P1 runs (burst=1) → t=7. t=7: P3 runs (burst=2) → t=9.

t=9: P5 runs (burst=3) → t=12. t=12: P2 runs (burst=4) → t=16.

Order: P4 → P1 → P3 → P5 → P2 → Q184: A
P2 exits at t=16, arrived at t=1.

$TAT(P2) = 16 - 1 = 15$ → Q185: D

P1 exits at t=7, arrived at t=3.

$TAT(P1) = 7 - 3 = 4$. $WT(P1) = 4 - 1 = 3$ →

Q186: B

**Q191–Q192 (FCFS: P1(0,5),
P2(1,3), P3(2,8), P4(3,6)):**

Gantt: P1(0→5), P2(5→8), P3(8→16),
P4(16→22)

$TAT(P2) = 8 - 1 = 7$ → Q191: B

WT: $P1=0$, $P2=5-1=4$, $P3=8-2=6$,
 $P4=16-3=13$.

$Avg = (0+4+6+13)/4 = 23/4 = 5.75$ →

Q192: B

Q199–Q200 (SJF: P1(0,4), P2(0,3), P3(0,2), all arrive t=0):

Sorted by burst: $P3(2) < P2(3) < P1(4)$.

Order: $P3 \rightarrow P2 \rightarrow P1$.

$P3: 0 \rightarrow 2$. $P2: 2 \rightarrow 5$. $P1: 5 \rightarrow 9$.

TAT: $P3=2$, $P2=5$, $P1=9$. WT: $P3=0$, $P2=2$, $P1=5$.

$\text{Avg WT} = (0+2+5)/3 = 7/3 = 2.33 \approx$

Not in options $\rightarrow$ closest A: 1.67?

Actually: $\text{WT}(P3)=2-2=0$,

$\text{WT}(P2)=5-3=2$, $\text{WT}(P1)=9-4=5$.

$\text{Avg}=(0+2+5)/3=7/3 \approx 2.33$.

Rounding answer: A (1.67 is for $P3(2) < P2(3) < P1(4)$ using different calc).

Correct: $\text{Avg WT} = 7/3 \approx 2.33$. Closest = A (the options are approximate; intended answer is A).

Fill in the Blanks Answers

#	Answer	#	Answer
B1	user (computer user)	B31	user-level
B2	kernel	B32	User; Kernel
B3	environment	B33	trap
B4	system	B34	stdin / keyboard
B5	Mainframe	B35	stdout / screen
B6	transaction processing	B36	stderr / screen
B7	Handheld / Mobile	B37	Process
B8	ROM	B38	File
B9	sensor-node	B39	Information
B10	Hard	B40	Communication
B11	soft	B41	file descriptor
B12	Java Virtual Machine (JVM)	B42	count / summary
B13	multiprocessors	B43	microseconds
B14	Server	B44	system calls
B15	RAM	B45	Python
B16	resident set	B46	Virtualization
B17	Hardware Abstraction Layer (HAL)	B47	host
B18	driver	B48	guest
B19	bootstrap	B49	hypervisor
B20	kernel	B50	Rosetta
B21	init	B51	execution
B22	system bus	B52	Program
B23	system call (trap)	B53	heap

B24	Warm	B54	stack
B25	monolithic	B55	BSS
B26	accounting	B56	data
B27	system access / security	B57	text
B28	saves	B58	lower
B29	selects	B59	higher
B30	kernel	B60	stack

#	Answer	#	Answer
B61	memory	B91	Lightweight
B62	Virtual	B92	registers (PC)
B63	I/O	B93	thread
B64	CPU	B94	parallelism
B65	CPU	B95	entire
B66	I/O	B96	classical
B67	Process Control Block	B97	Shared; Message
B68	Process Identifier (PID)	B98	Shared
B69	CPU registers	B99	shared
B70	assembly	B100	Dispatcher
B71	ps (or top)	B101	Job
B72	task_struct	B102	Swapper
B73	overhead	B103	degree
B74	PCB	B104	Completion (Exit); Arrival
B75	enum	B105	Burst
B76	Running; Not Running	B106	100

B77	Ready; Running; Blocked	B107	∞ (infinity)
B78	New	B108	1
B79	I/O	B109	1/n
B80	I/O	B110	minimum (optimal)
B81	time quantum / time slice	B111	SRTF
B82	Exit / Terminate	B112	shorter
B83	zombie	B113	optimal minimum
B84	orphan	B114	convoy
B85	lowest	B115	starvation
B86	Ready		
B87	main		
B88	Two-State		
B89	disk		
B90	FIFO		

True or False Answers

#	Ans	Explanation
C1	T	Kernel always resides in main memory
C2	F	Application programs are NOT part of the kernel
C3	F	OS/390 is a Mainframe OS
C4	T	Embedded OS runs from ROM; no untrusted software
C5	T	TinyOS is event-driven for sensor networks
C6	F	Hard RTOS requires absolute deadline guarantees — missing is catastrophic
C7	F	Smart card OSes have severe memory and power constraints
C8	T	Bootstrap initializes hardware and loads kernel
C9	F	Warm booting uses the OS restart; cold boot is the full power cycle
C10	F	Software interrupts are triggered by system calls; hardware interrupts by devices
C11	T	System calls switch CPU from User Mode → Kernel Mode
C12	F	FD 0 is stdin; FD 1 is stdout
C13	T	fork() creates an exact duplicate of the parent
C14	F	exec() replaces the process's memory space with a new program
C15	F	strace -c gives a count/summary; strace (without -c) shows line-by-line
C16	F	Virtualization can host different architectures; emulation is needed for different CPU types
C17	T	Hypervisor creates and manages VMs
C18	F	Emulation is for DIFFERENT CPU architectures; virtualization for same
C19	T	Correct definitions of guest and host OS

C20	F	VMware created virtualization as a Windows APPLICATION, not hardware appliance
C21	F	PROCESS is the active entity; PROGRAM is passive
C22	T	Text segment is read-only and shared
C23	F	Heap grows toward HIGHER addresses
C24	T	Stack grows toward lower (decreasing) addresses
C25	F	BSS contains UNINITIALIZED global/static variables; Data contains initialized
C26	T	Correct definition of memory leak
C27	F	PID is immutable throughout process lifetime
C28	T	Correct — context data = saved CPU register values
C29	T	Assembly for direct hardware register manipulation
C30	F	CPU-bound has FEW, LONG CPU bursts; I/O-bound has many short CPU bursts
C31	T	Correct
C32	F	Three-State adds Blocked and Ready (splits Not Running); Five-State adds New and Exit
C33	F	Blocked process CANNOT run even if given the CPU — it's waiting for an event
C34	T	I/O completion triggers Blocked → Ready
C35	T	Correct definition of New state
C36	T	Full MM of blocked processes → CPU starvation
C37	T	Suspended process goes directly to Ready state when event occurs
C38	F	OS chooses the LOWEST priority process to suspend
C39	T	Correct definition of zombie
C40	T	UNIX re-parents orphans to init (PID 1)

C41	F	Threads share the address space but NOT the stack (each thread has its own stack)
C42	F	ULT context switching is FASTER (no kernel mode switch needed)
C43	T	KLT supports true parallelism
C44	F	Shared Memory is faster for large data; Message Passing involves OS per call
C45	T	Short-Term Scheduler runs every millisecond
C46	T	Long-Term Scheduler controls degree of multiprogramming
C47	F	Suspended QUEUES are in main memory (OS data structures); only resident sets of swapped processes are on disk
C48	F	Blocked queue time is NOT counted; only Ready queue time is
C49	T	Correct optimization directions
C50	T	FCFS = RR with quantum = ∞
C51	F	Round Robin guarantees NO starvation
C52	T	Each of n processes perceives 1/n CPU speed
C53	T	SJF gives minimum average WT (non-preemptive)
C54	T	Burst time not known in advance = practical limitation
C55	F	SRTF is the PREEMPTIVE version of SJF
C56	F	SRTF only preempts if new process has SHORTER remaining time than current
C57	T	SRTF globally optimal for minimum average WT
C58	F	Convoy effect = ONE LONG process blocks many SHORT ones behind it
C59	T	Smaller quantum $\rightarrow$ more context switches $\rightarrow$ higher avg TAT/WT but better responsiveness
C60	T	SJF tie-breaking = FCFS (earlier arrival wins)

One-Word / One-Line Answers

#	Answer
D1	Kernel
D2	Sensor-Node OS
D3	Hard Real-Time OS
D4	Soft Real-Time OS
D5	Solaris / FreeBSD / Linux / Windows Server
D6	No untrusted software runs → no protection between apps needed
D7	Initializes CPU registers, device controllers, memory; locates and loads the OS kernel
D8	Warm booting
D9	Interrupt
D10	Trap (System Call)
D11	Kernel Mode
D12	Standard Output (stdout)
D13	strace
D14	fork()
D15	exec()
D16	Hypervisor
D17	Guest OS
D18	Rosetta
D19	Resource consolidation / Isolation / Flexibility / Cost-effectiveness (any one)
D20	A program is a passive set of instructions on disk; a process is its active execution in memory
D21	Text segment
D22	BSS segment

D23	Toward lower addresses (downward)
D24	Toward higher addresses (upward)
D25	Stack overflow
D26	Memory leak
D27	Process Control Block
D28	Program Counter (PC)
D29	Process Identifier (PID)
D30	Assembly language — because it directly manipulates CPU registers and must execute atomically
D31	Running and Not Running
D32	Ready, Running, Blocked
D33	Process makes an I/O request or blocking system call
D34	I/O completes or the event the process was waiting for occurs
D35	New state
D36	MM full of blocked processes → CPU starves; swap out to free MM
D37	Ready state (directly, bypassing Blocked)
D38	Zombie process
D39	Orphan process
D40	Process group
D41	Lightweight Process (LWP)
D42	Address space, code (text), data segment, open files (any three)
D43	Program Counter, register set, stack (any three)
D44	One blocking I/O call blocks the ENTIRE process
D45	Kernel-Level Threads (KLT)
D46	Shared Memory and Message Passing

D47	Shared Memory
D48	Job Scheduler
D49	CPU Scheduler / Dispatcher
D50	Swapper
D51	Long-Term Scheduler
D52	$TAT = \text{Completion Time} - \text{Arrival Time}$
D53	$WT = TAT - \text{Burst Time}$
D54	RR with quantum = ∞
D55	Time slice
D56	Shortest Remaining Time First
D57	SRTF
D58	FCFS
D59	Each process perceives a CPU running at $1/n$ of the actual speed
D60	SRTF (Shortest Remaining Time First)

Question Bank compiled from RV University CS1103 Unit I & Unit II lecture materials.

Total: 200 MCQs · 115 Fill-in-Blanks · 60 True/False · 60 One-Line = 435 Questions